

# Chương 8: Phân loại Ảnh với Mạng Tích chập (Image Classification with ConvNets)

Học Sâu (Deep Learning)

Đại học Đông Á | August 14, 2026

Gemini Notebook

# Nội dung chương




Nền tảng của Mạng Tích chập (CNN)

Thực hành: Bài toán Phân loại Chó & Mèo

Chống Overfitting bằng Data Augmentation

Chuyển giao Học tập (Transfer Learning)

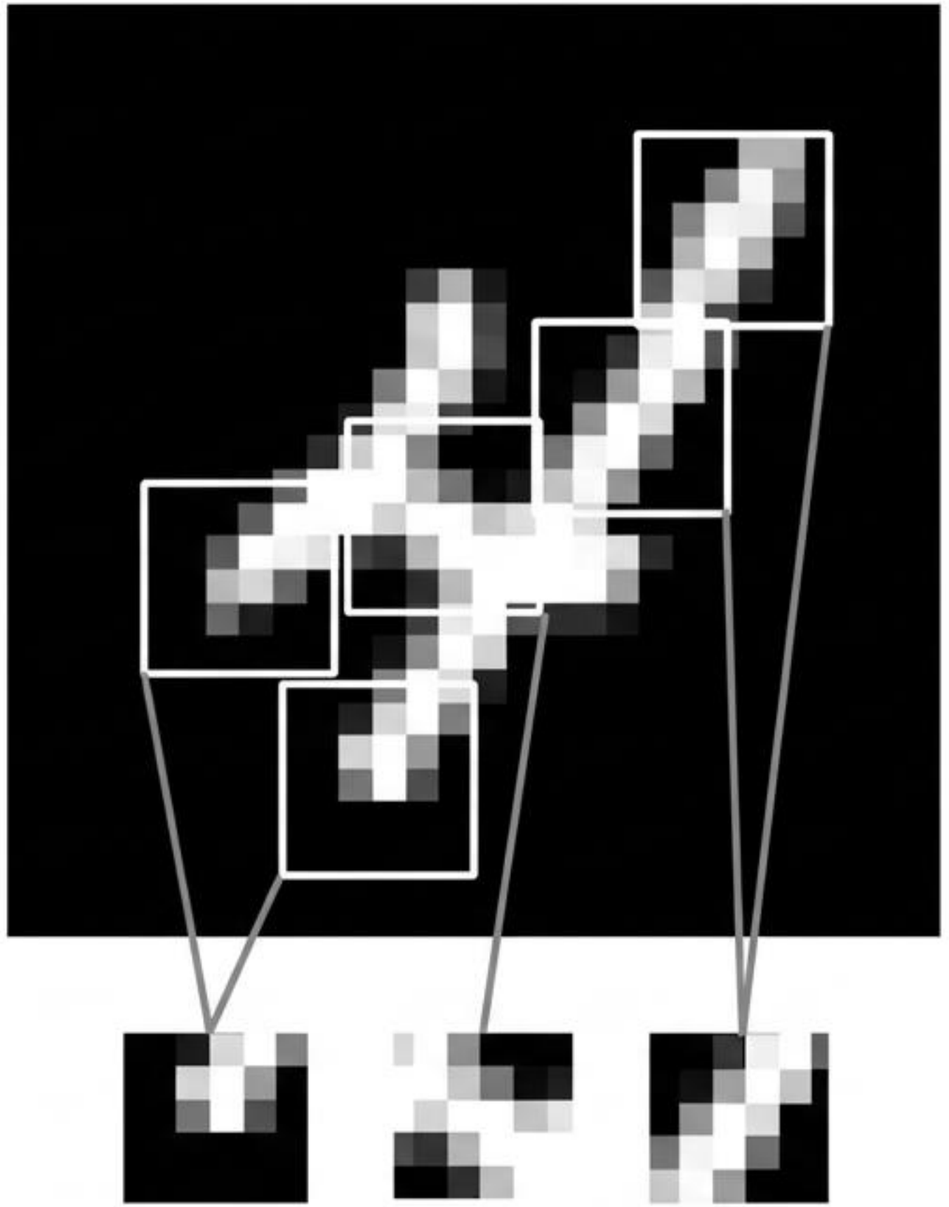
Tinh chỉnh (Fine-Tuning)

Tổng kết

Gemini Notebook

# Tại sao lại là Tích chập (Convolution)?

- Mạng Dense truyền thống: Tìm kiếm các mẫu toàn cục (Global Patterns) trên toàn bộ bức ảnh.
- Mạng Tích chập (ConvNets): Tìm kiếm các mẫu cục bộ (Local Patterns) như cạnh, góc, kết cấu.
- Tính bất biến dịch chuyển (Translation Invariance): Nếu CNN học được hình dáng con con mắt ở góc trái ảnh, nó vẫn có thể nhận ra con mắt đó ở góc phải ảnh. Dense Layer thì không!

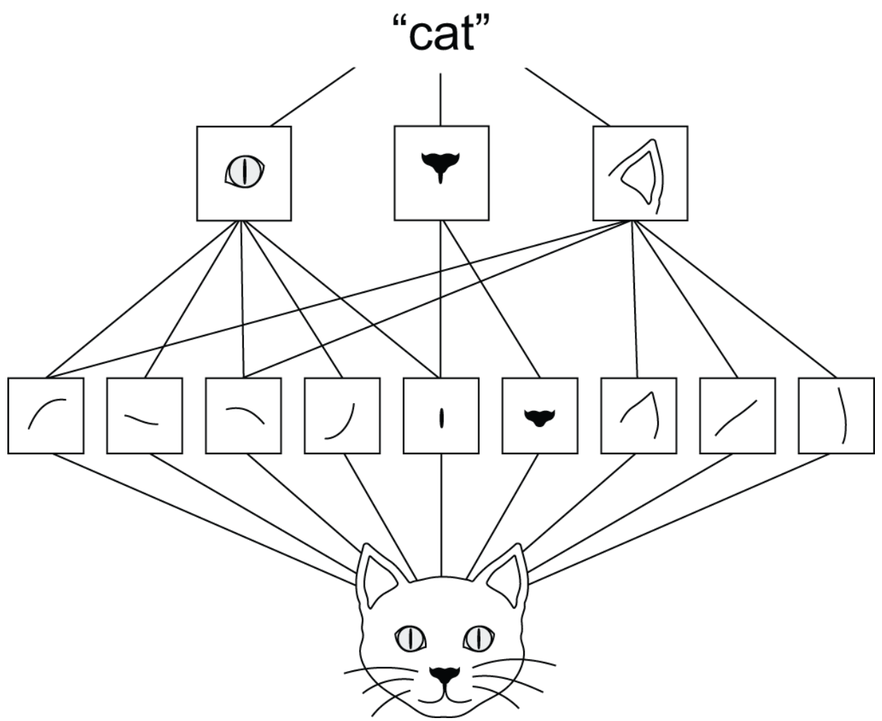


**Hình 8.1:** Local Patterns vs Global Patterns

Gemini Notebook

# Hệ thống Phân cấp Thị giác (Visual Hierarchy)

- CNN học theo một hệ thống phân cấp tự nhiên:
  - **Lớp đầu tiên**: Học các đặc trưng đơn giản (Cạnh, Góc).
    - **Lớp giữa**: Ghép các cạnh thành bộ phận (Mắt, Mũi, Lưỡi).
      - **Lớp cuối:** Tổng hợp thành vật thể hoàn chỉnh (Con mèo, Con chó).
- Điều này **giống hệt** cách vỏ não thị giác (Visual Cortex) của con người hoạt động.




Gemini Notebook



# Mạng CNN cơ bản: Mã nguồn

Cấu trúc một mạng phân loại ảnh đơn giản bao gồm việc xen kẽ các lớp **Conv2D** và **MaxPooling2D**.

```
import keras
from keras import layers

inputs = keras.Input(shape =(28, 28, 1))
x = layers.Conv2D(filters =64, kernel_size =3, activation='relu')(inputs)
x = layers.MaxPooling2D(pool_size =2)(x)
x = layers.Conv2D(filters =128, kernel_size =3, activation='relu')(x)
x = layers.MaxPooling2D(pool_size =2)(x)
x = layers.Conv2D(filters =256, kernel_size =3, activation='relu')(x)
x = layers.GlobalAveragePooling2D ()(x)
outputs = layers.Dense (10, activation='softmax')(x)

model = keras.Model(inputs=inputs , outputs=outputs)
```

Gemini Notebook

# Kiểm tra quá trình nén không gian (Model Summary)

**Kích thước không gian** (chiều rộng × chiều cao) giảm dần, trong khi chiều sâu (số kênh) tăng dần.

```
>>> model.summary ()
Model: 'functional'
...
Layer (type)                                 Output Shape                              Param #
=================================================================
input_layer (InputLayer)                     (None , 28, 28, 1)                        0
conv2d (Conv2D)                              (None , 26, 26, 64)                       640
max_pooling2d (MaxPooling2D)                 (None , 13, 13, 64)                       0
conv2d_1 (Conv2D)                            (None , 11, 11, 128)                      73856
max_pooling2d_1 (MaxPooling2D)               (None , 5, 5, 128)                        0
conv2d_2 (Conv2D)                            (None , 3, 3, 256)                        295168
global_average_pooling2d                     (None , 256)                              0
dense (Dense)                                (None , 10)                               2570
...
```

Gemini Notebook

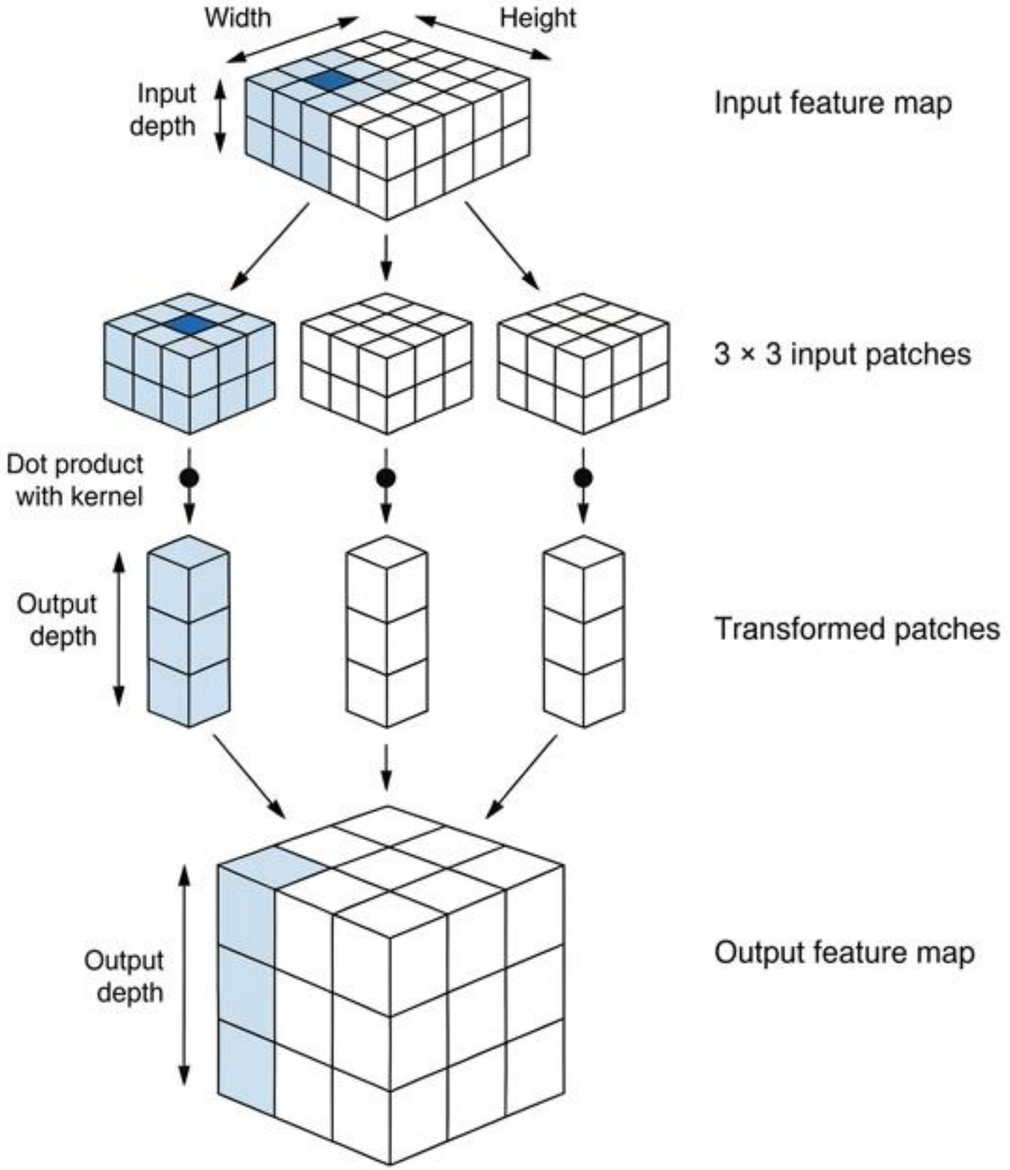
# Toán học của Phép Tích chập (Dot Product)

Toán tử Tích chập 2D được định nghĩa:

$$S(i,j) = (I * K)(i,j)$$
$$= \sum_m \sum_n I(i-m, j-n) K(m,n)$$

$I$: Ma trận ảnh đầu vào.
$K$: Bộ lọc (Kernel / Filter).

Bản chất là trượt bộ lọc K trên toàn bộ ảnh và tính Tích vô hướng (Dot Product).

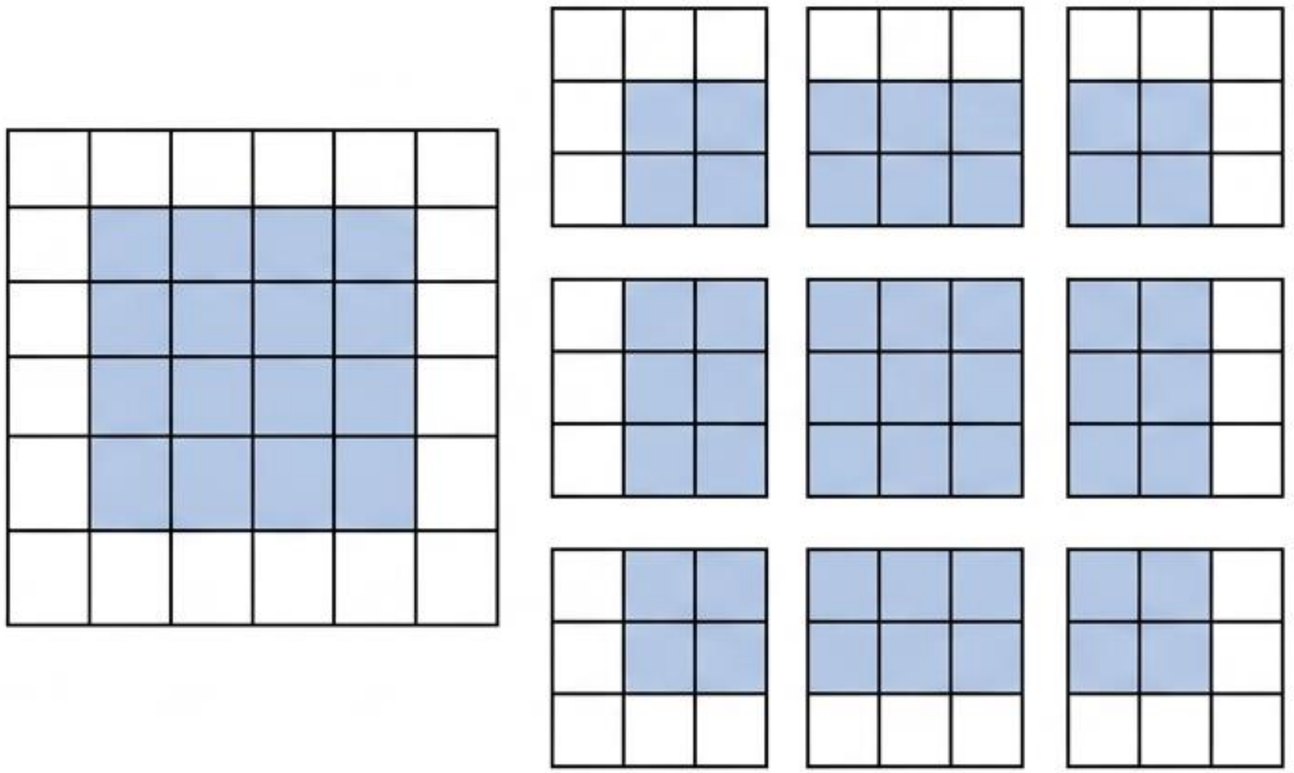



*Hình 8.3: Phép trượt nhân chập cơ bản*

Gemini Notebook

# Trích xuất Phân mảnh (Patches)

- Nếu có ảnh đầu vào kích thước 5×5, và sử dụng Bộ lọc kích thước 3×3.
- Bộ lọc sẽ cắt ra các phân mảnh (patches) có cùng kích thước 3×3.
- Khớp với mỗi phân mảnh, phép Tích vô hướng sẽ sinh ra duy nhất một con số (Scalar).



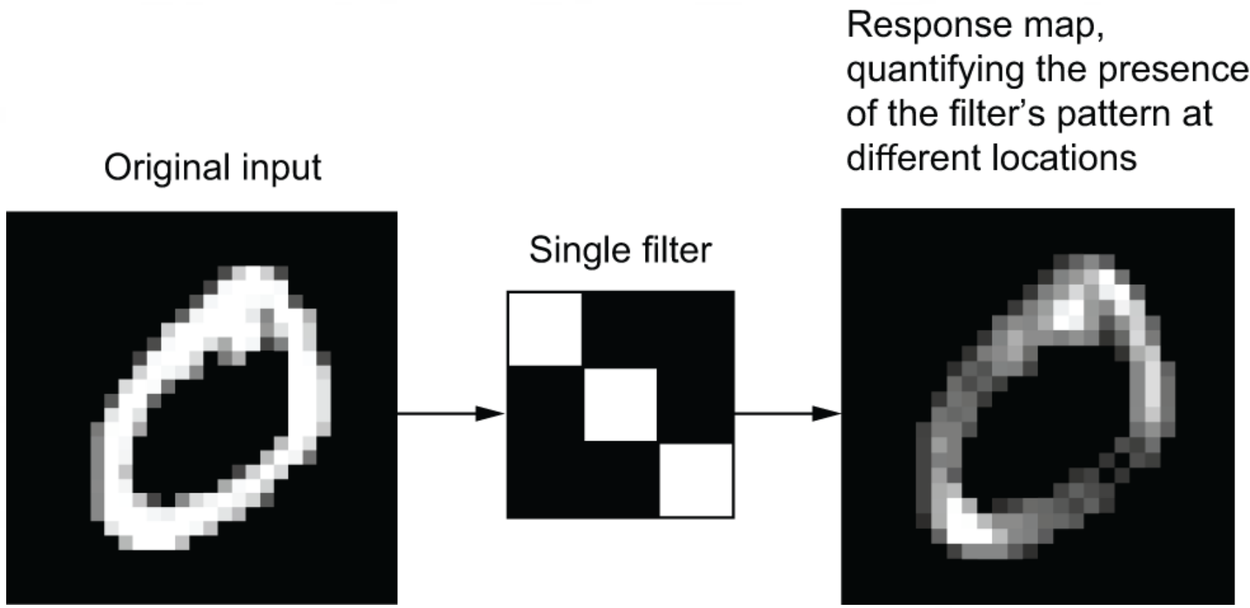
Hình 8.4: Trích xuất các patches 3×3

Gemini Notebook

# Bản đồ Phản hồi (Response Map)



- Mỗi Bộ lọc (Filter) đại diện cho một "Khái niệm" (Vd: Khái niệm "có mắt mèo trong hình không?").
- Kết quả của quá trình tích chập tạo ra một Bản đồ phản hồi (Response Map).
- Điểm nào sáng trên bản đồ nghĩa là Bộ lọc đã "nhìn thấy" mẫu mà nó đang tìm kiếm tại vị trí đó.

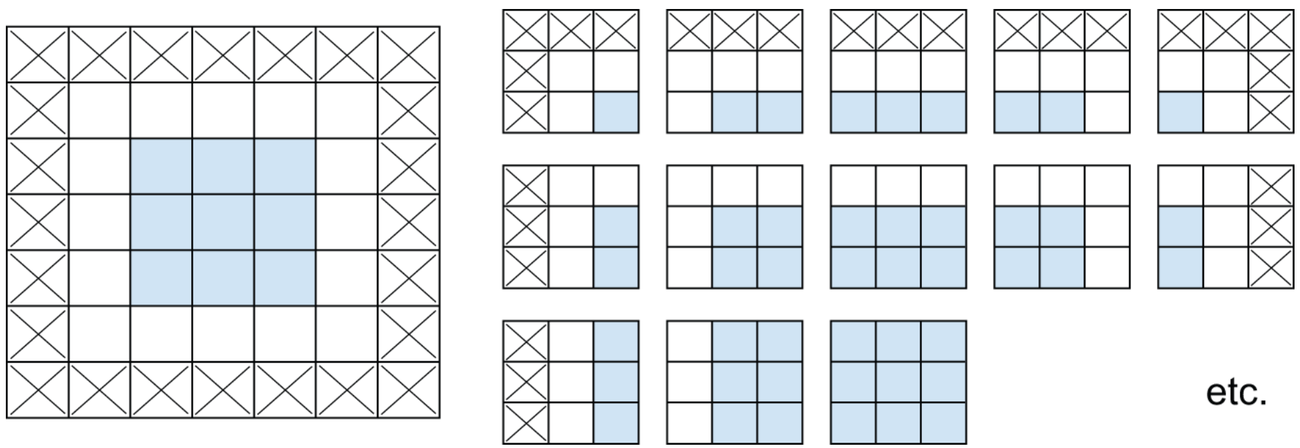



**Hình 8.5:** Một bản đồ phản hồi đặc trưng

Gemini Notebook

# Kỹ thuật Lấp đầy (Padding)

- Vấn đề: Kích thước ảnh luôn bị co hẹp lại sau khi Tích chập (Từ 5×5 xuống 3×3). Lớp biên của ảnh gốc bị bỏ qua rất nhiều.
- Giải pháp (Padding): Thêm các viền số 0 (Zero-padding) bao quanh ảnh để bảo toàn kích thước sau khi quét.
- Tham số Keras: padding="same" (Bảo toàn) hoặc padding="valid" (Không lấp đầy).

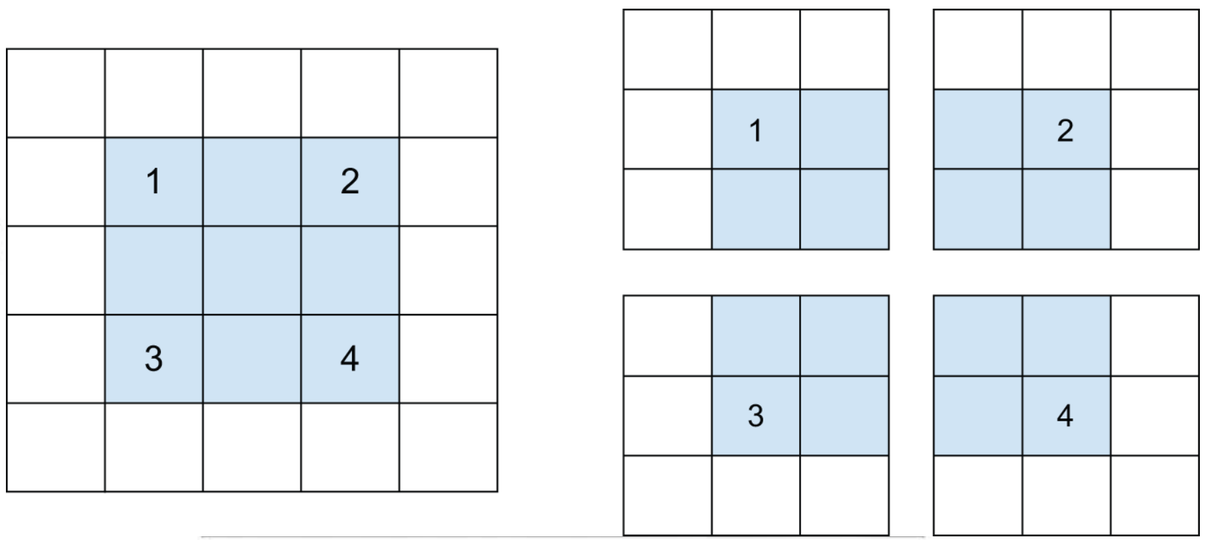



**Hình 8.6**: Minh họa Zero Padding

Gemini Notebook

# Bước nhảy (Strides)

- Stride: Khoảng cách (số pixel) mà Bộ lọc trượt đi sau mỗi nhịp. Mặc định là strides=1.
- Nếu strides=2, bộ lọc sẽ nhảy cóc qua 1 pixel. Kích thước bản đồ đầu ra sẽ giảm đi chính xác một nửa (Downsampling).
- Lưu ý: Trong thực tế ta thường dùng MaxPooling để giảm kích thước thay vì Strides.




*Hình 8.7: Bộ lọc trượt với Strides bằng 2*

Gemini Notebook

# Tập Dữ liệu Kaggle Dogs vs Cats



- Bài toán thi đấu nổi tiếng năm 2013 trên Kaggle.
- Cung cấp 25,000 ảnh chó và mèo nhiều kích cỡ, màu sắc, bối cảnh.
- Mục tiêu thực hành: Ta sẽ trích xuất ra một tập con siêu nhỏ (Chỉ 2,000 ảnh cho Training) để giả lập kịch bản thiếu dữ liệu rất biến trong doanh nghiệp.



*Hình 8.8: Mẫu ảnh từ bộ dữ liệu*

Gemini Notebook

# Tiền xử lý ảnh với Keras

Với những dữ liệu lớn, việc đọc tất cả vào RAM là bất khả thi.
image_dataset_from_directory tạo một Data Generator lấy ảnh trực tiếp từ ổ cứng theo
generator lấy ảnh trực tiếp từ ổ cứng theo từng lô.

```
from keras.utils import image_dataset_from_directory

train_dataset = image_dataset_from_directory(
    'dogs_vs_cats_small/train',
    image_size =(180 , 180),
    batch_size =32
)

validation_dataset = image_dataset_from_directory(
    'dogs_vs_cats_small/validation',
    image_size =(180 , 180),
    batch_size =32
)
```

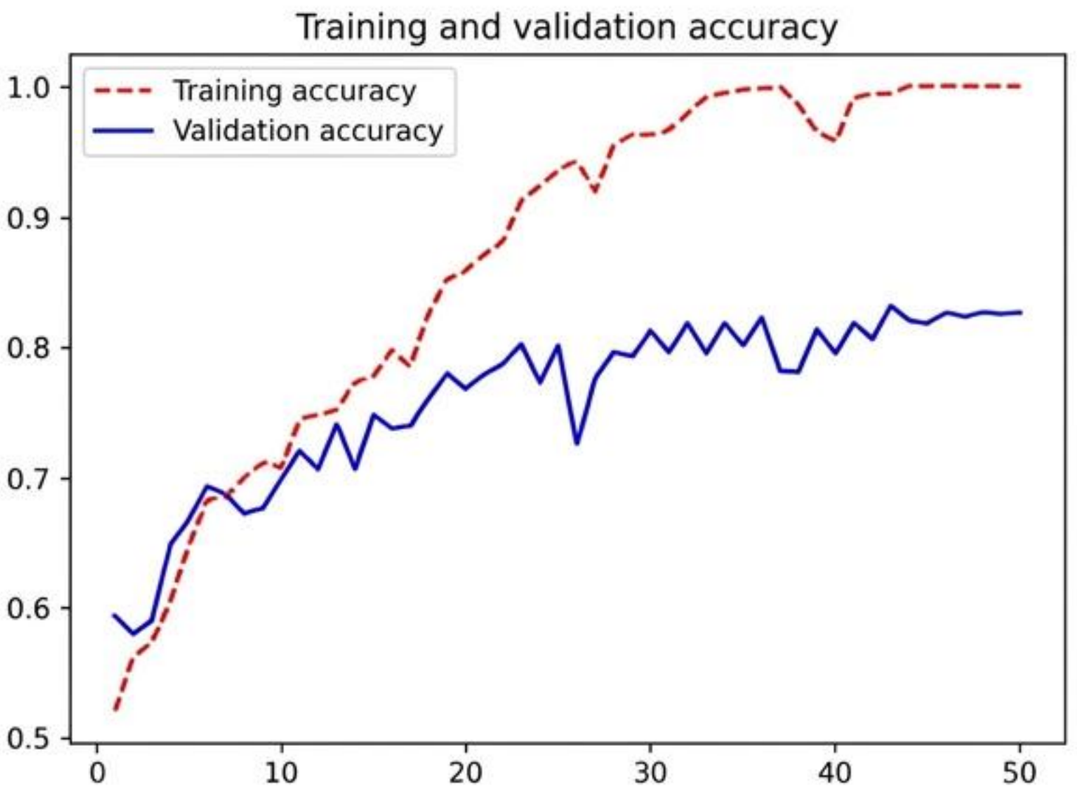
Gemini Notebook

# Huấn luyện CNN cơ bản (Từ con số 0)

- Thiết kế mạng với 4 khối Conv2D Conv2D + MaxPooling2D.
- Kết quả: Validation Accuracy đạt khoảng 70-75% rồi chững lại.



- Khoảng cách giữa đường Train (Xanh) và Validation (Đứt nét) nới rộng nhóng nới rộng nhanh chóng → Dấu hiệu Overfitting điển hình.

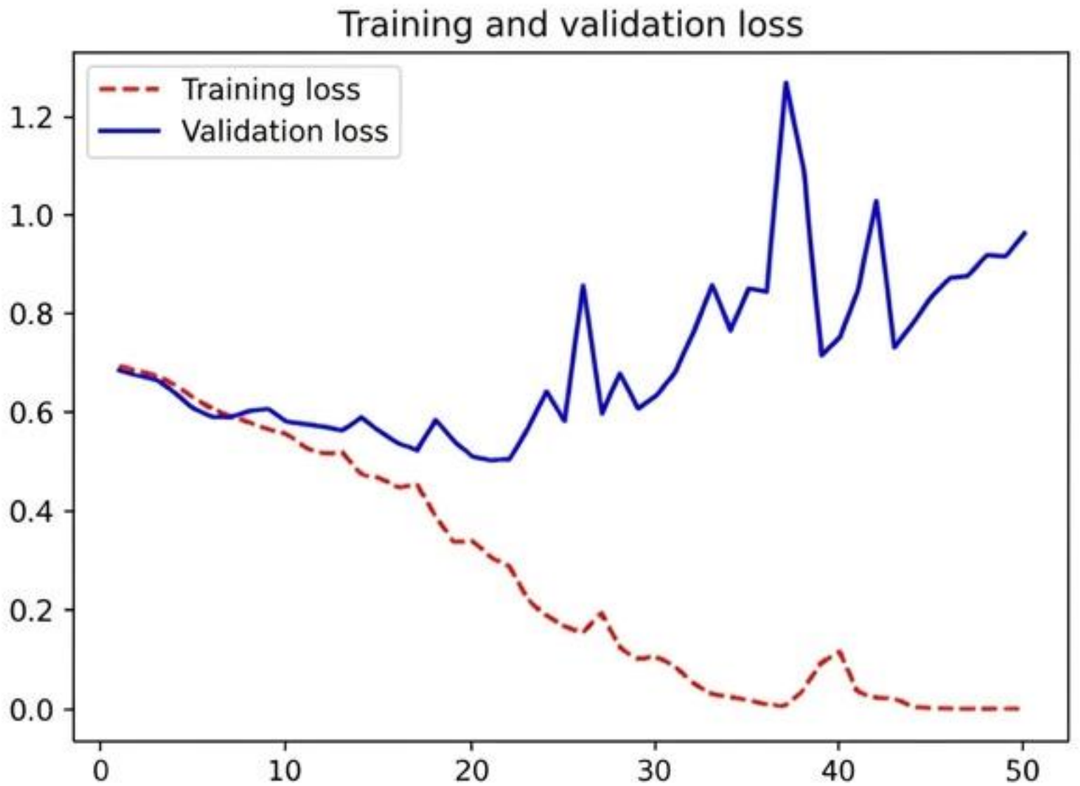



*Hình 8.9: Accuracy Mô hình cơ bản*

Gemini Notebook

# Phân tích hàm Loss cơ bản

- Hàm mất mát Training liên tục giảm tiến về 0. Mô hình đang "thuộc lòng" (Memorizing) dữ liệu học.
- Hàm mất mát Validation không thể giảm xuống dưới ngưỡng 0.8 và bắt đầu bật tăng ngược trở lại sau Epoch 10.
- Kết luận: Do dữ liệu huấn luyện quá ít (2000 ảnh), mạng CNN không thể khái quát hóa. Ta cần kỹ thuật mạnh hơn.




*Hình 8.10: Loss Mô hình cơ bản*

Gemini Notebook



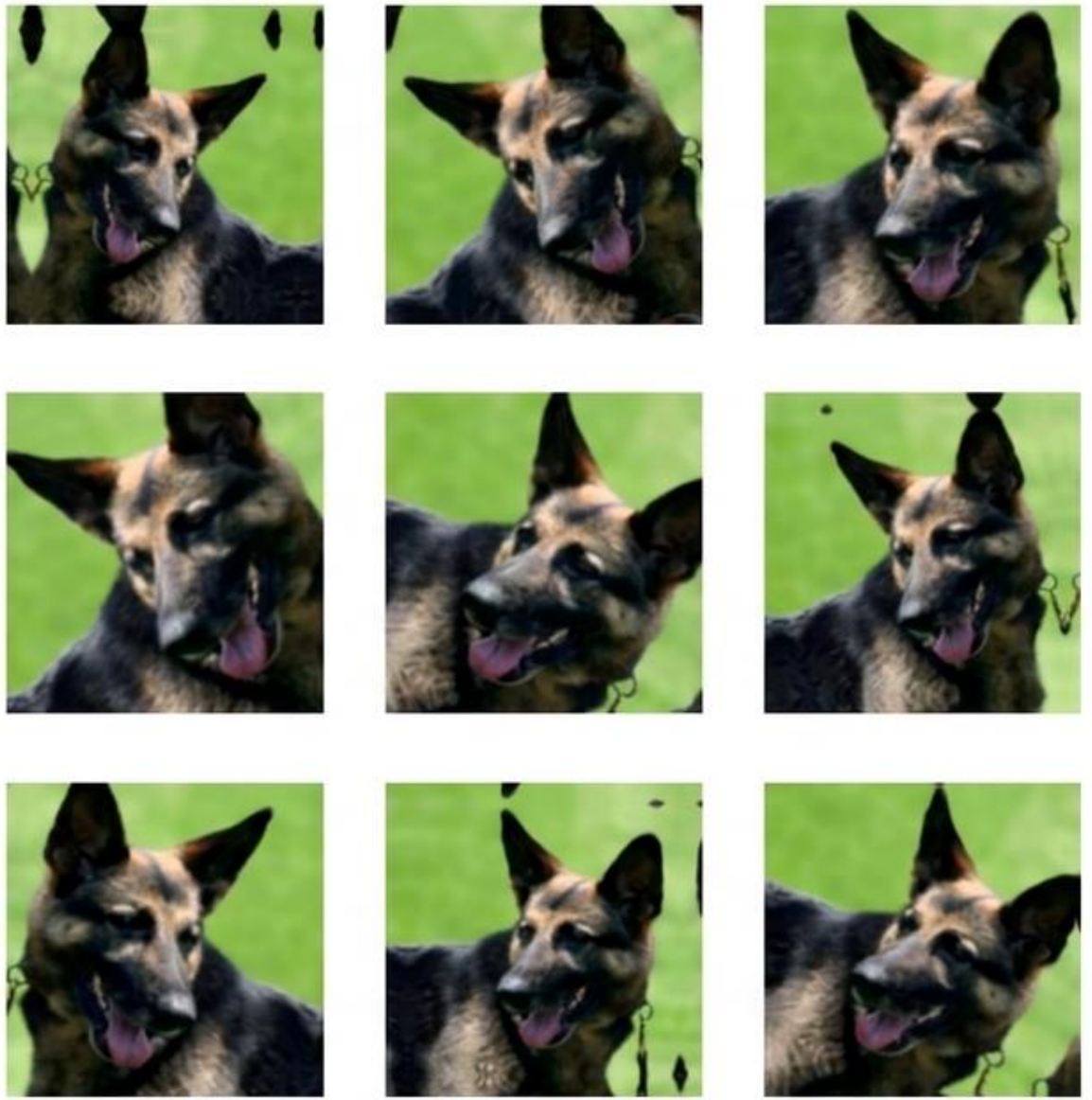
# Cải thiện Hiệu suất Mô hình Học Sâu

- Kỹ thuật Tăng cường Dữ liệu (Data Augmentation)
- Trích xuất Đặc trưng (Feature Extraction) & Fine-Tuning

Gemini Notebook

# Khái niệm Kỹ thuật Tăng cường Dữ liệu

- Kỹ thuật Tăng cường Dữ liệu (Data Augmentation)
- Không có thêm dữ liệu? Hãy tự tạo ra dữ liệu mới từ dữ liệu cũ!
- Áp dụng các phép biến đổi ngẫu nhiên: Xoay (Rotation), Dịch chuyển (Translation), Thu phóng (Zoom), Lật ngang (Horizontal Flip).
- Mô hình sẽ không bao giờ nhìn thấy cùng một bức ảnh 2 lần, buộc nó phải học các đặc trưng bất biến.



*Hình 8.11: Cùng một con mèo nhưng sinh ra nhiều biến thể*

Gemini Notebook



# Data Augmentation: Mã nguồn

Keras cung cấp sẵn các Layer tăng cường dữ liệu. Các lớp này có thể được gắn thẳng vào mô hình.

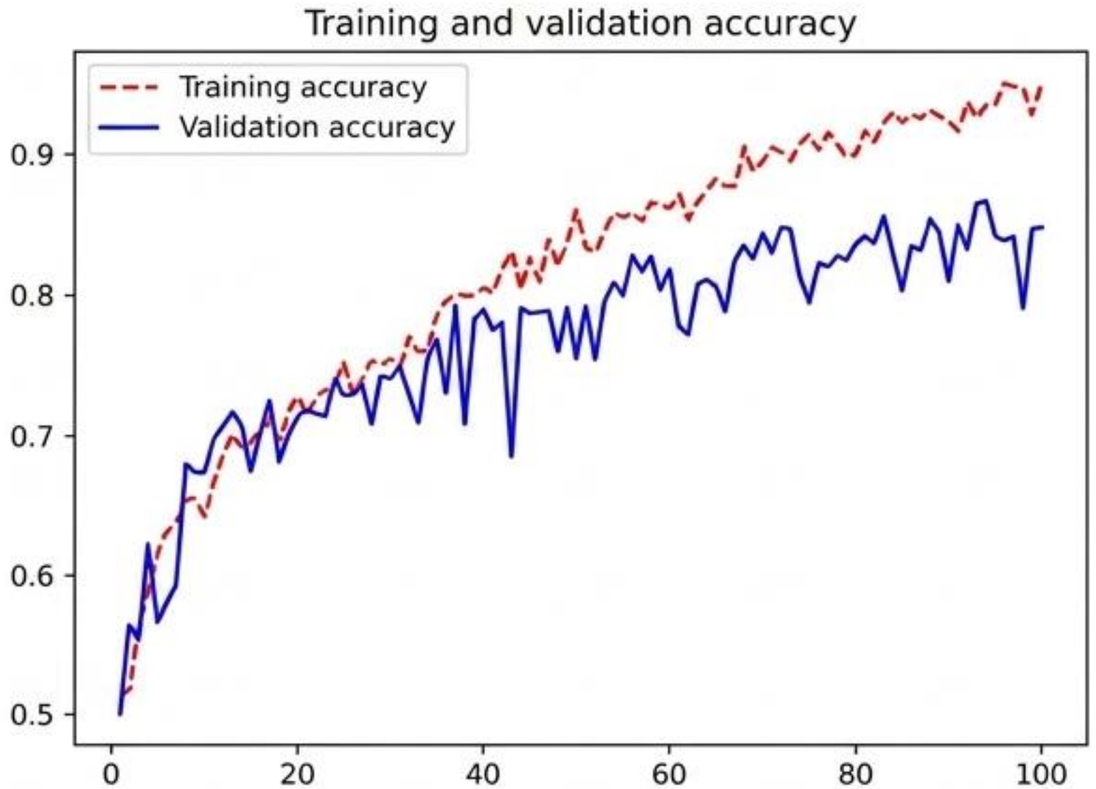
```python
data_augmentation = keras.Sequential(
  [
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.2),
  ]
)

# Insert augmentation layer before the main ConvNets
inputs = keras.Input(shape =(180 , 180, 3))
x = data_augmentation(inputs)
x = layers.Rescaling(1./255)(x)
x = layers.Conv2D(filters =32, kernel_size =3, activation="relu")(x)
# ... Add more Convolutional Blocks ...
```

⚠️ Lưu ý: Data Augmentation tự động bị Vô hiệu hóa (Deactivated) trong quá trình Testing / Validation.

Gemini Notebook

# Đánh giá Accuracy sau Augmentation




***Hình 8.12:*** *Accuracy có Data Augmentation*



- Validation Accuracy đã tăng từ 70% lên hơn 80%.
- Quan trọng hơn: Đường Validation (Đứt nét) bám sát đường Training (Xanh).
- Mô hình hoàn toàn không còn hiện tượng Overfitting, và chúng ta có thể tiếp tục huấn luyện lâu hơn.

Gemini Notebook

# Đánh giá Loss sau Augmentation & Giới hạn

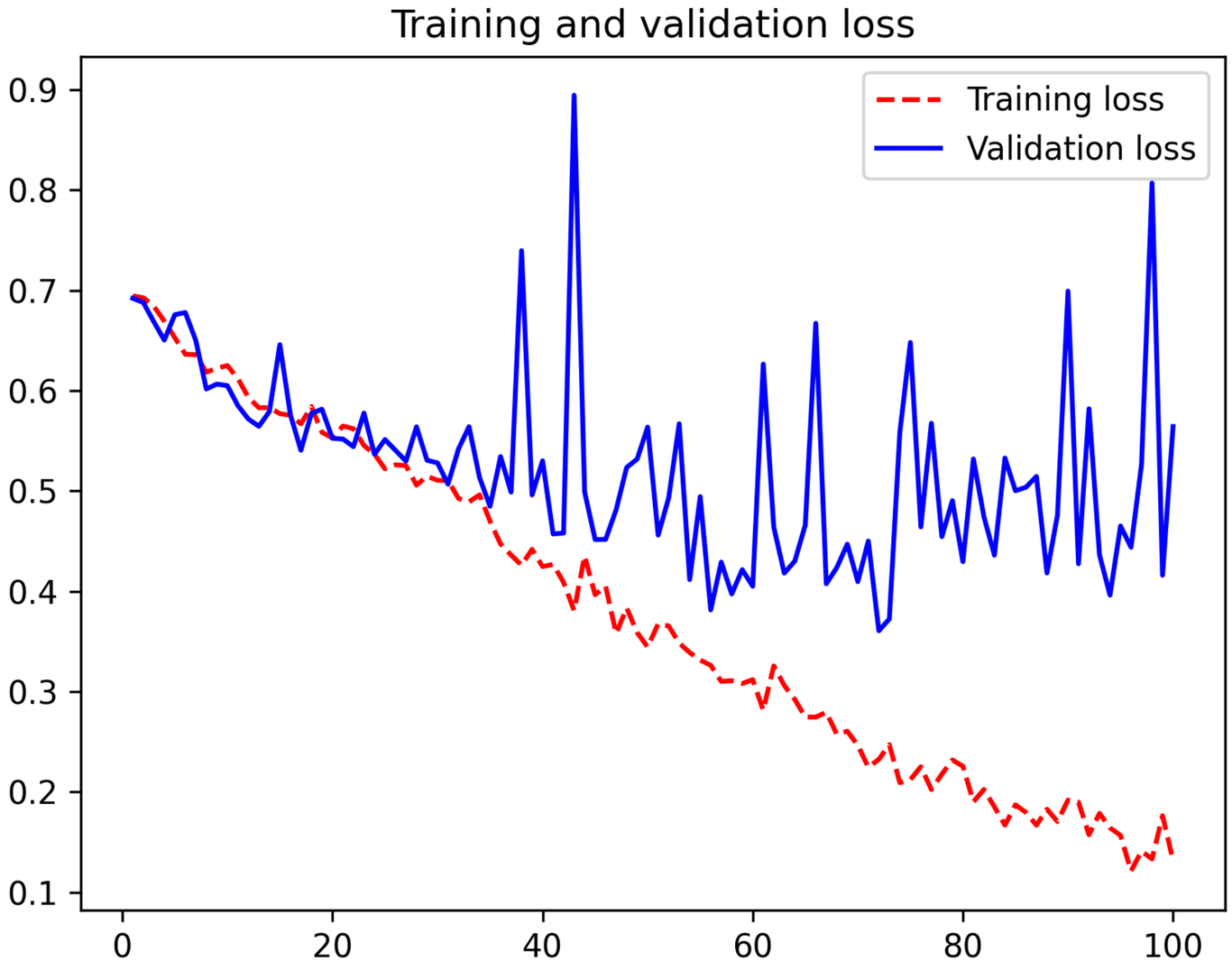


**Đánh giá Loss sau Augmentation**

- Đồ thị Loss không còn bị nảy ngược lên như trước.
- Kỹ thuật Augmentation được tính toán song song trên CPU/GPU qua hàm tf.data pipeline mà không làm chậm quá trình Train.



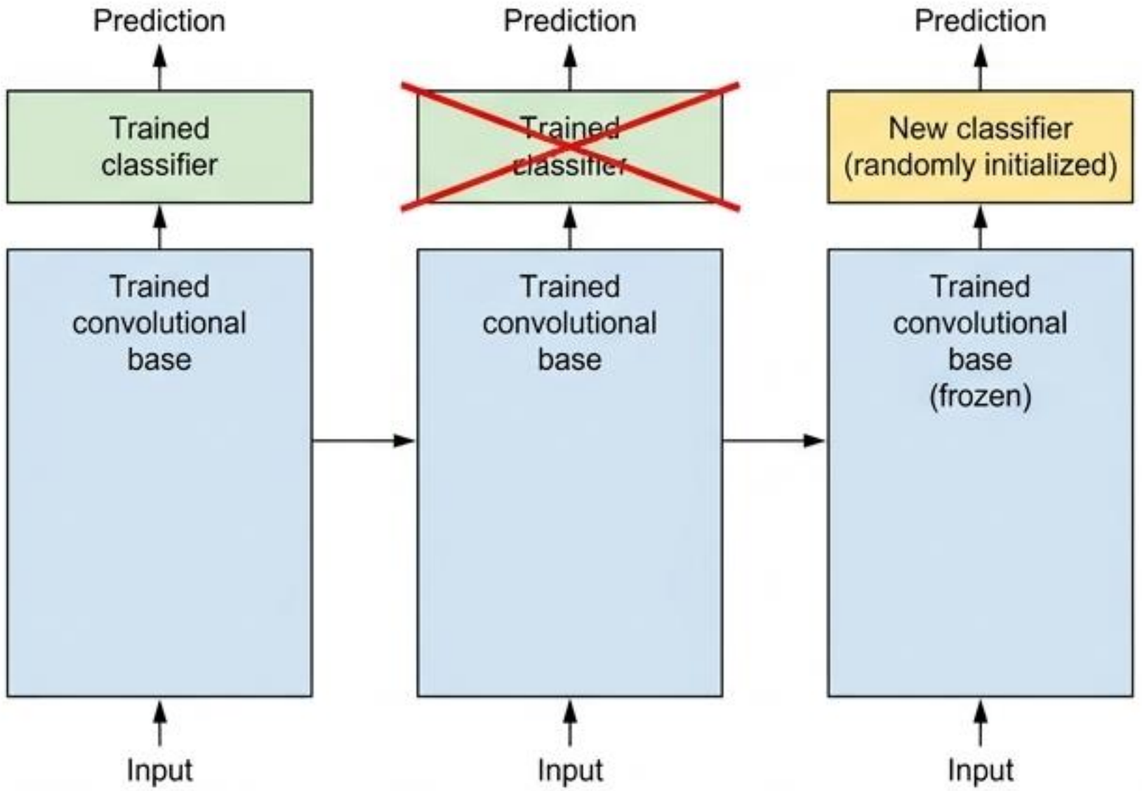
**Giới hạn:** Ta vẫn đang bị kẹt ở mốc 82%. Để vượt ngưỡng này, ta cần sức mạnh từ Transfer Learning.




**Hình 8.13**: Loss có Data Augmentation

Gemini Notebook

# Trích xuất Đặc trưng (Feature Extraction)




**Hình 8.14**: Thay thế bộ phân loại (Swapping FC)

- **Khái niệm**: Tái sử dụng một mô hình cực lớn (VGG16) đã được huấn luyện trên 1.4 triệu bức ảnh của ImageNet.
- **Kiến trúc**: Giữ nguyên Khối Tích chập (Convolutional Base) của VGG16.
- **Cắt bỏ** phần đầu Phân loại (Fully Connected Classifier) cũ của VGG16, và **Gắn** lõi Phân loại mới của chúng ta vào.

Gemini Notebook

# Feature Extraction - Mã nguồn



**Feature Extraction**: Mã nguồn (Sử dụng VGG16)

Khởi tạo mô hình VGG16, sau đó “Đóng băng” (Freeze) toàn bộ trọng số của Base.

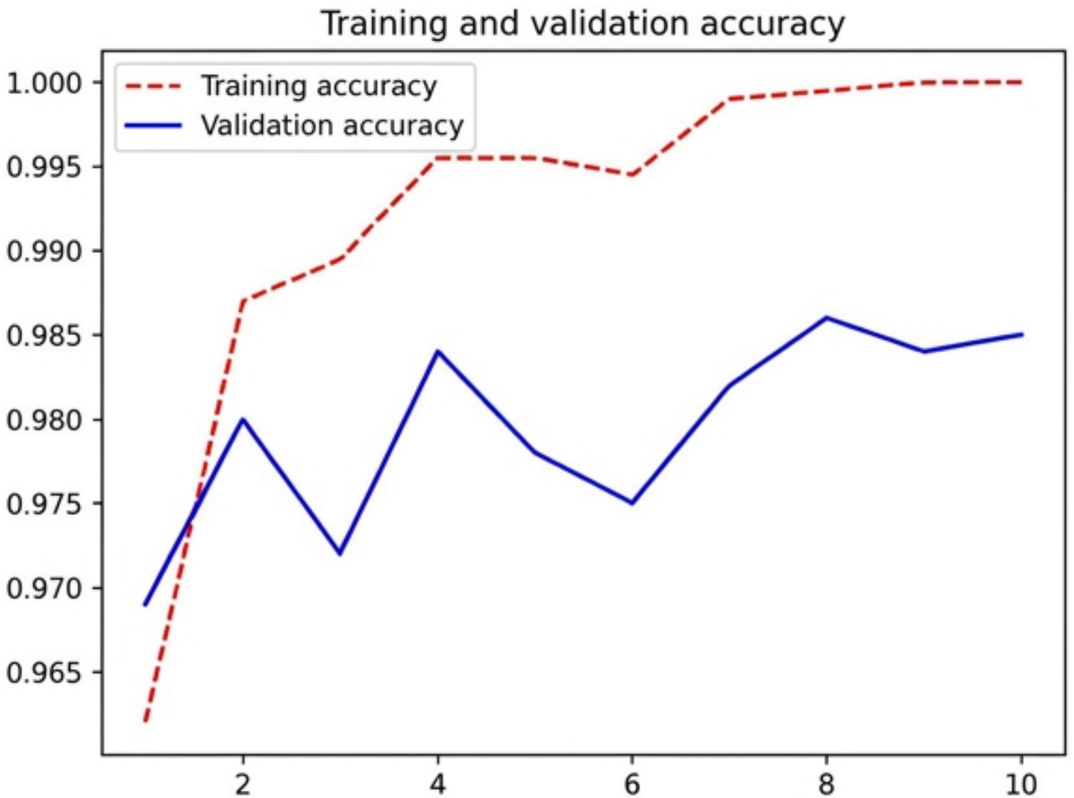
```
# Load pre -trained model without the top dense layers (include_top=False)
conv_base = keras.applications.vgg16.VGG16(
    weights="imagenet",
    include_top=False ,
    input_shape =(180, 180, 3)
)

# Freeze the base!
conv_base.trainable = False

# Attach our custom classifier
inputs = keras.Input(shape =(180 , 180, 3))
x = conv_base(inputs)
x = layers.Flatten ()(x)
x = layers.Dense (256, activation="relu")(x)
x = layers.Dropout (0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs , outputs)
```

Gemini Notebook

# Accuracy với Feature Extraction Cơ bản



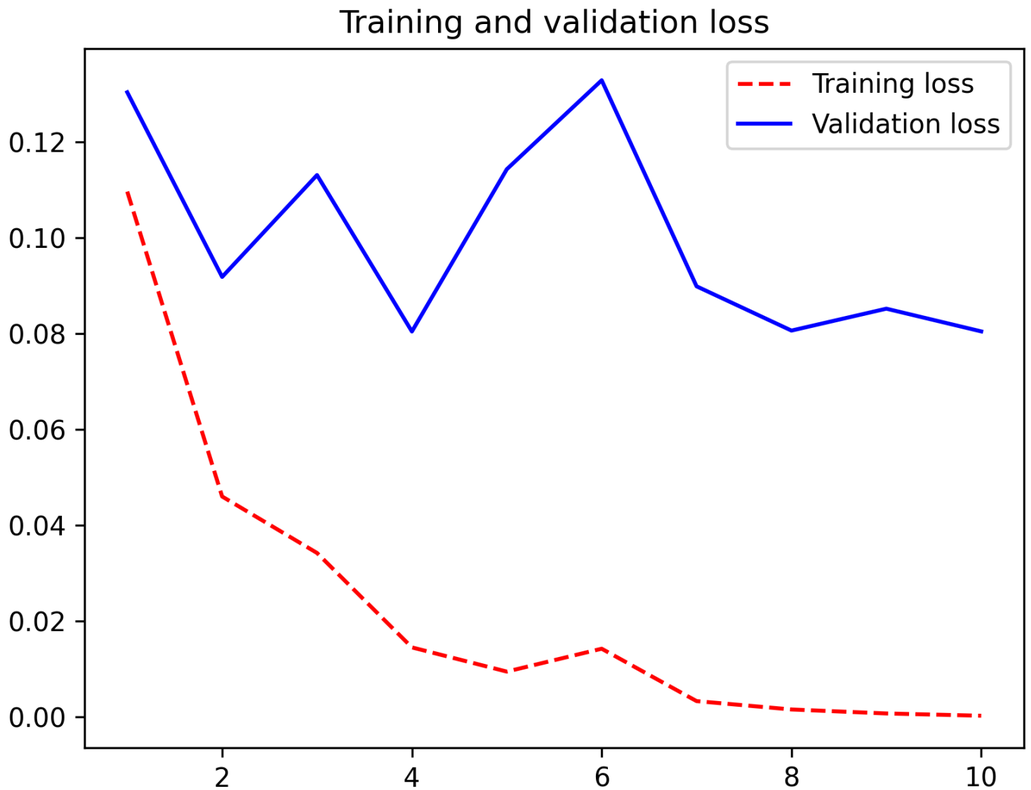

Hình 8.15: Accuracy Feature Extraction



- Bằng cách đóng băng (Freeze) VGG16 Base, ta chỉ huấn luyện lớp Dense mới thêm vào.
- Kết quả đáng kinh ngạc: Validation Accuracy vọt lên 90%.
- Sự mạnh mẽ của ImageNet đã giúp trích xuất các đặc trưng mắt mèo/mũi chó cực kỳ tinh xảo.

Gemini Notebook

# Loss & Vấn đề Overfitting




**Hình 8.16:** Loss Feature Extraction (Bị Overfitting)



## Loss với Feature Extraction Cơ bản

- Tuy nhiên, vì tốc độ học quá nhanh, đồ thị Loss lại tiếp tục báo hiệu Overfitting cực kỳ nặng ngay từ Epoch thứ 5.
- **Tại sao?** Vì ta chưa **áp dụng Data Augmentation** cho quy trình này.

Gemini Notebook

# Mô hình Liên hoàn - FE + Data Augmentation

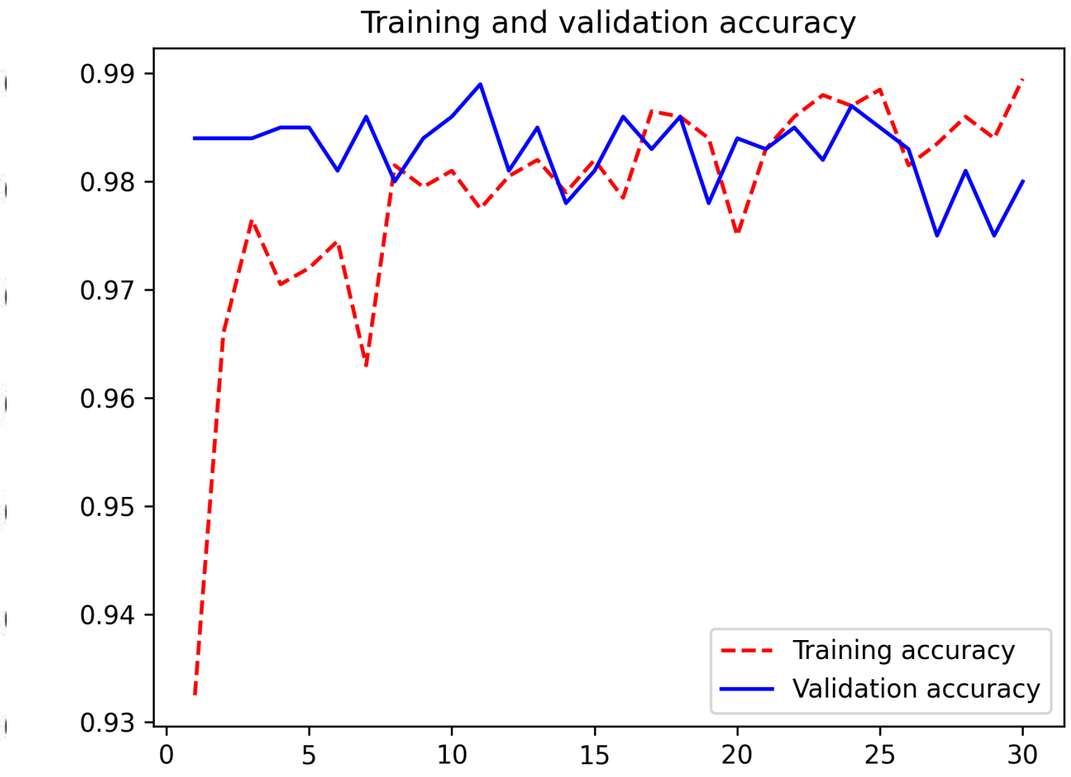


- Để giải quyết bài toán Overfitting, ta kết hợp Data Augmentation vào chuỗi tính toán.
- Đây là kỹ thuật yêu cầu GPU mạnh vì phải chạy forward-pass qua toàn bộ Base nhiều lần.

```
inputs = keras.Input(shape =(180, 180, 3))
# 1. Apply Augmentation
x = data_augmentation(inputs)
# 2. Preprocess specifically for VGG16
x = keras.applications.vgg16.preprocess_input(x)
# 3. Extract features via Frozen Base
x = conv_base(x)
# 4. Custom classifier
x = layers.Flatten ()(x)
x = layers.Dense (256)(x)
x = layers.Dropout (0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs , outputs)
```

Gemini Notebook

# Accuracy: Feature Extraction + Data Augmentation



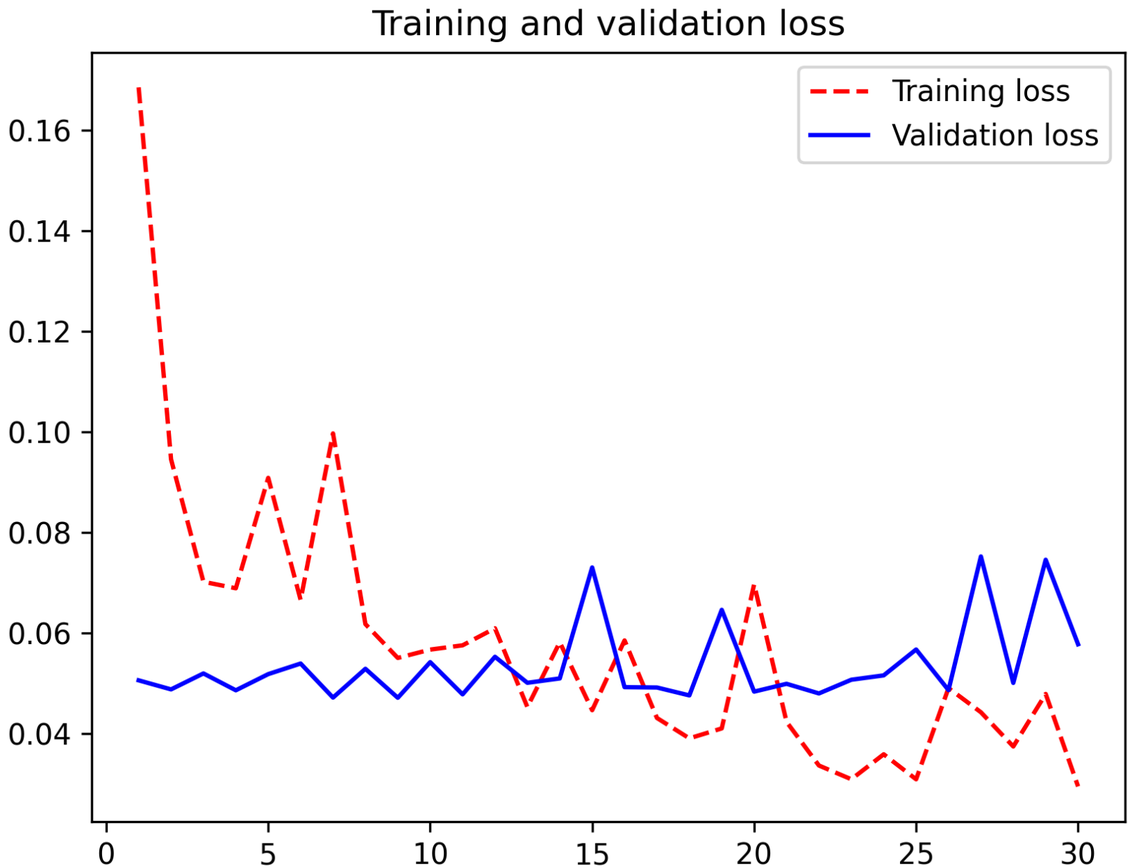

*Hình 8.17: Accuracy FE + Data Augmentation*



- Ta thiết lập một kiến trúc liên hoàn: Lớp Augmentation → VGG16 Base (Đóng băng) → Lớp Dense (Huấn luyện).
- Kết quả: Mượt mà hoàn hảo. Validation bám sát Training và không còn Overfitting. Đạt mốc 96%.

Gemini Notebook

# Loss: Feature Extraction + Data Augmentation




**Hình 8.18:** Loss FE + Data Augmentation



- Đồ thị Loss cực kỳ đẹp.
- Quá trình này đòi hỏi máy phải có card đồ họa (GPU), vì VGG16 phải chạy dự đoán lại từ đầu qua mỗi bức ảnh biến đổi ngẫu nhiên.

Gemini Notebook

# Quy trình Fine-Tuning



Để ép hiệu suất mô hình đến giới hạn cực đại:

1. Gắn bộ phân loại tùy chỉnh lên trên một mô hình cơ sở (VGG16).
2. Đóng băng toàn bộ mô hình cơ sở (Freeze Base).
3. Huấn luyện bộ phân loại tùy chỉnh.
4. Mở khóa (Unfreeze) một vài lớp Tích chập trên cùng của Base.
5. Huấn luyện đồng thời cả bộ phân loại và các lớp Tích chập vừa mở khóa với Tốc độ học cực nhỏ (Rất nhỏ).

Fine-Tuning: Mã nguồn giải mã các lớp cuối. Chỉ mở khóa Block cuối cùng của VGG16 (block 5).

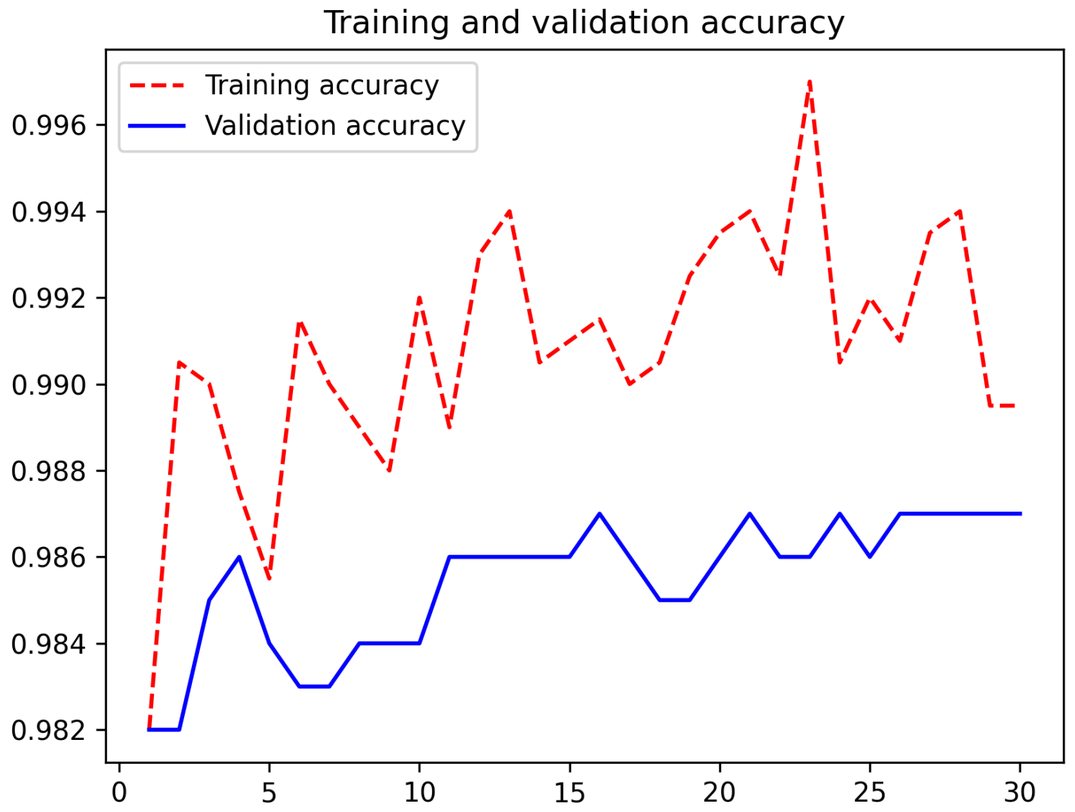
```python
conv_base.trainable = True

# Freeze all layers EXCEPT the ones from 'block5_conv1' onwards
set_trainable = False
for layer in conv_base.layers:
    if layer.name == 'block5_conv1':
        set_trainable = True
    if set_trainable:
        layer.trainable = True
    else:
        layer.trainable = False

# Re-compile model with a VERY SMALL learning rate
model.compile(
    loss='binary_crossentropy',
    optimizer=keras.optimizers.RMSprop(learning_rate =1e-5),
    metrics =['accuracy']
)
```

Gemini Notebook

# Kết quả Accuracy sau Fine-Tuning



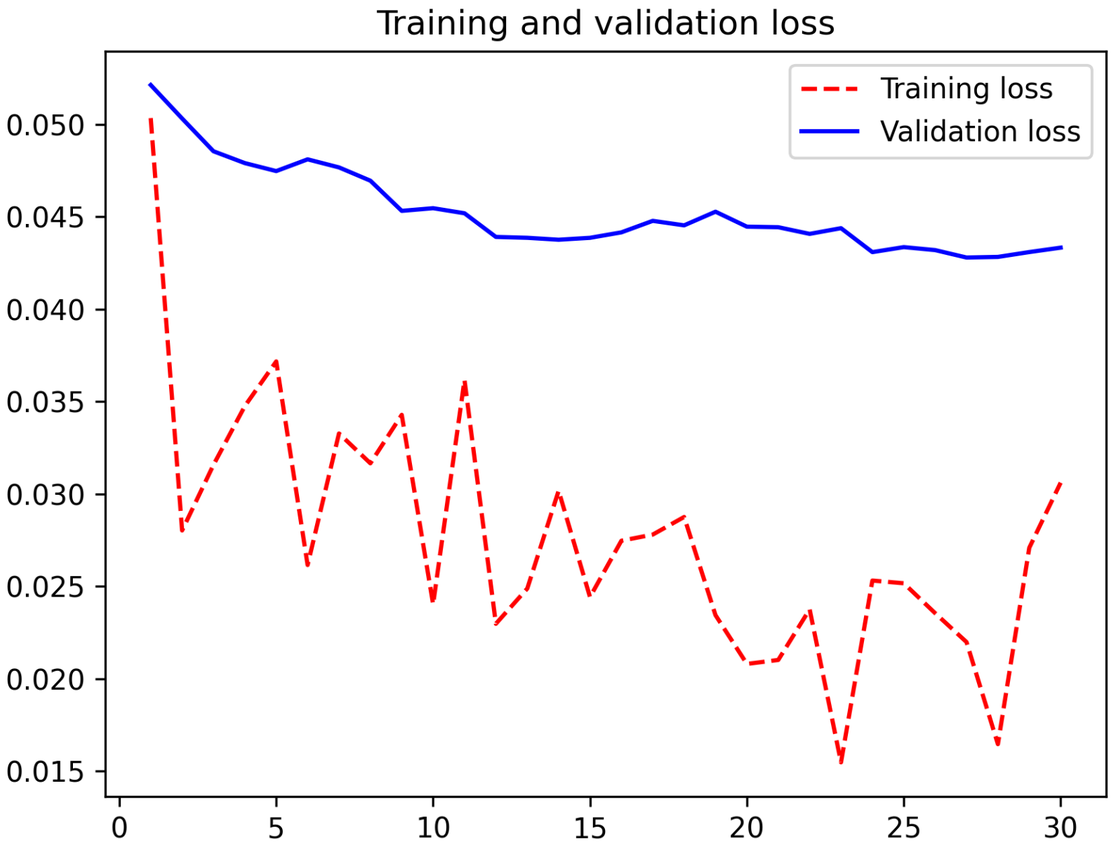

Hình 8.19: Accuracy sau Fine-Tuning



- Bằng cách nới lỏng nhẹ trọng số của khối Block 5 (VGG16), mô hình tự điều chỉnh các đặc trưng bậc cao sao cho khớp hoàn hảo với chó/mèo.
- Độ chính xác vọt lên 97.5%. Đánh bại hoàn toàn mô hình cơ sở ban đầu.

Gemini Notebook

# Kết quả Loss sau Fine-Tuning




**Hình 8.20:** Loss sau Fine-Tuning




- Đồ thị Loss có thể biến động nhẹ (nhiễu), nhưng giá trị trung bình trượt dài (Moving Average) cho thấy sự ổn định.
- Mặc dù chỉ dùng 2000 ảnh (so với hàng chục ngàn ảnh của người thắng giải Kaggle), ta vẫn đạt kết quả xuất chúng nhờ Transfer Learning.

Gemini Notebook

# Tổng kết Chương 8



Mạng **Tích chập (CNN)** vượt trội trong xử lý Ảnh nhờ khả năng học các Mẫu cục bộ bất biến với sự dịch chuyển.



Các khối cơ bản: Phép nhân Tích chập **Conv2D** và Giảm độ phân giải **MaxPooling2D**.



**Data Augmentation** là vũ khí tối thượng chống lại **Overfitting** khi thiếu hụt Dữ liệu.



Tái sử dụng Tri thức bằng **Feature Extraction** và **Fine-Tuning (VGG16)** giúp đạt hiệu suất State-of-the-art chỉ với vài ngàn tấm ảnh.

Gemini Notebook